Professor(a)	Projeto	Período	Data
WESLEY SOBREIRA	PIPELINE DE DADOS	2º	

Grupo

ROTEIRO DE ATIVIDADES

AULA 09 - PIPELINE DE PONTA A PONTA: EXTRACT, TRANSFORM E LOAD

Leiam atentamente as ORIENTAÇÕES abaixo, antes de iniciar a atividade.
1. A atividade é em grupo e está dividida em blocos. Cada bloco tem passos a executar e perguntas de verificação. **2.** Os passos devem ser feitos na ordem em que aparecem. Não avancem para o bloco seguinte sem responder as perguntas de verificação do bloco atual. **3.** As respostas devem ser escritas em células de texto do próprio notebook do pipeline e precisam citar os números reais da base escolhida pelo grupo. **4.** Ao final, o grupo entrega o notebook, o README, o requirements.txt e a pasta data com as três camadas (bronze, silver e gold).

Quadro geral da aula

Bloco	Atividade
Bloco 1	Formar o grupo e escolher o dataset
Bloco 2	Conhecer a fonte e definir a pergunta de negócio
Bloco 3	Extract: construir a Bronze
Bloco 4	Transform: construir a Silver
Bloco 5	Load: construir a Gold
Bloco 6	Load: modelagem dimensional (fato e dimensões)
Bloco 7	Costurar o pipeline e provar que ele se reconstrói
Bloco 8	Carga incremental
Bloco 9	Segunda Gold e visualização
Bloco 10	Parametrizar o pipeline
Bloco 11	Documentação e reprodutibilidade

BLOCO 1 | ESCOLHA DO DATASET

Passos

[ ] Formem grupos de 3 ou 4 pessoas e escrevam os nomes dos integrantes na primeira célula do notebook.
[ ] Leiam a seção Datasets sugeridos, no final deste roteiro, e separem 2 candidatos.
[ ] Confiram os critérios de escolha para cada candidato e baixem uma amostra de cada um.
[ ] Escolham 1 dataset. Quem já tem a base do Projeto Aplicado pode usá-la, desde que atenda aos critérios.

Perguntas de verificação (respondam no notebook)

P1. Qual dataset o grupo escolheu e por que ele foi preferido ao segundo candidato?
P2. Quantas fontes (arquivos, tabelas ou endpoints) ele possui e quantas linhas tem a maior delas?
P3. Que problema de qualidade vocês já enxergaram apenas olhando uma amostra?

BLOCO 2 | CONHECER A FONTE E DEFINIR A PERGUNTA

Passos

[ ] Carreguem cada fonte e executem shape, dtypes, head() e a contagem de nulos por coluna.
[ ] Identifiquem a chave que liga as fontes (se houver mais de uma) e a coluna de data (se houver).
[ ] Escrevam, no topo do notebook, UMA pergunta de negócio em uma frase. Ela será respondida pela camada Gold.
[ ] Desenhem no papel o fluxo fonte, Bronze, Silver e Gold, indicando quais fontes e colunas cada camada usa.

Perguntas de verificação (respondam no notebook)

P1. Que tabela agregada responde à pergunta do grupo? Nessa tabela, uma linha representa o quê?
P2. Quais fontes ou colunas NÃO são necessárias para a pergunta? Por que deixá-las de fora é uma decisão da etapa de Extract?
P3. Algum nulo encontrado pode estar carregando informação em vez de indicar erro? Se sim, qual é o nulo e o que ele significa?

BLOCO 3 | EXTRACT: CONSTRUIR A BRONZE

Passos

[ ] Criem as pastas data/bronze, data/silver e data/gold.

[ ] Escrevam a função extract(destino_bronze), que lê cada fonte necessária e grava na Bronze sem corrigir nada.

[ ] Acrescentem a cada arquivo da Bronze os metadados de controle: data e hora da extração e origem do dado.

[ ] Fonte em arquivo: leiam só as colunas necessárias. Fonte em banco: leiam com sqlite3 e SQL. Fonte em API: tratem a paginação e façam o código parar com mensagem clara se o status da resposta for diferente de 200.

[ ] Executem extract() duas vezes seguidas e anotem o número de linhas da Bronze após cada execução.

Perguntas de verificação (respondam no notebook)

P1. Quantas linhas e colunas tem cada arquivo da Bronze? Os números batem com os da fonte original?

P2. O que aconteceu com o número de linhas na segunda execução? Se mudou, por que mudou e como corrigir?

P3. Por que a Bronze não deve receber nenhuma correção, mesmo que o grupo já conheça os problemas do dado?

P4. Se a fonte sair do ar amanhã, o que do pipeline continua funcionando?

BLOCO 4 | TRANSFORM: CONSTRUIR A SILVER

Passos

[ ] Escrevam transform(origem_bronze, destino_silver), lendo somente da Bronze e nunca da fonte original.

[ ] Padronizem tipos (datas como datetime, números como numéricos) e nomes de colunas.

[ ] Tratem duplicados, contando as linhas antes e depois.

[ ] Tratem os nulos coluna a coluna, decidindo entre remover, preencher ou manter. Registrem cada decisão em uma célula de texto no formato: decidimos X porque Y.

[ ] Tratem valores inválidos (negativos, datas impossíveis, categorias inconsistentes) e contem quantos registros cada regra afetou.

[ ] Se houver mais de uma fonte, façam a junção e comparem a soma da principal coluna numérica antes e depois do join.

[ ] Apliquem o filtro de escopo que a pergunta de negócio exigir, contando quantos registros ele descartou.

Perguntas de verificação (respondam no notebook)

P1. Quantas linhas tem a Bronze e quantas tem a Silver? Explique a diferença usando cada uma das decisões tomadas.

P2. A soma da coluna principal bateu antes e depois da junção? Se não bateu, o que foi duplicado ou perdido?

P3. Qual decisão de limpeza mais alterou o resultado final? Como o grupo mediu isso?

P4. Algum nulo foi mantido de propósito? Justifique.

P5. Se uma regra de limpeza mudar amanhã, qual camada precisa ser reprocessada e qual não precisa? Por quê?

BLOCO 5 | LOAD: CONSTRUIR A GOLD

Passos

[ ] Escrevam load(origem_silver, destino_gold), que agrega a Silver para responder à pergunta do grupo.

[ ] Calculem pelo menos 2 métricas na agregação. Para contar entidades distintas, usem nunique e não count.

[ ] Gravem a Gold em Parquet.

[ ] Gravem também a Gold em uma tabela SQLite com if_exists="replace" e executem uma consulta SQL que responda à pergunta.

[ ] Comparem o tamanho em disco da Silver e da Gold.

Perguntas de verificação (respondam no notebook)

P1. Qual é a resposta da pergunta de negócio? Mostrem o resultado (o top 5 ou o valor encontrado).

P2. Quantas linhas tem a Gold e quantas tem a Silver? O que se perde na agregação e por que isso é aceitável?

P3. O resultado da consulta SQL é igual ao do pandas? Provem com uma comparação em código.

P4. Por que gravar a Gold em dois formatos? Quem consome cada um deles?

P5. O que aconteceria na segunda execução se o grupo tivesse usado if_exists="append"?

BLOCO 6 | LOAD: MODELAGEM DIMENSIONAL

Passos

[ ] Escrevam em uma frase o grão da tabela fato do grupo, no formato: cada linha representa um(a) ____.

[ ] Identifiquem na Silver quais colunas descrevem o contexto (quem, o quê, onde, quando) e quais são medidas numéricas.

[ ] Criem a dim_tempo, com uma linha por data e colunas de ano, mês, trimestre e dia da semana.

[ ] Criem pelo menos mais uma dimensão (por exemplo produto, cliente, zona ou região), com uma linha por membro e uma chave substituta própria, diferente da chave original. Se o dataset tiver poucas dimensões naturais, criem a dim_tempo e uma segunda dimensão definida pelo grupo, justificando a escolha.

[ ] Criem a tabela fato apenas com as chaves substitutas das dimensões e as medidas numéricas, sem repetir texto descritivo.

[ ] Gravem as dimensões e a fato em Parquet e em SQLite, com chave primária nas dimensões e chaves estrangeiras na fato.

[ ] Respondam à pergunta de negócio do grupo com um JOIN entre a fato e as dimensões e comparem o resultado com o da Gold agregada.

[ ] Acrescentem a construção do modelo dimensional à função load(), para que pipeline() o gere automaticamente.

Perguntas de verificação (respondam no notebook)

P1. Qual é o grão da tabela fato? Como o grupo garantiu que ele foi respeitado, ou seja, que não há duplicidade dentro do grão?

P2. A soma da principal medida na fato bate com a soma na Silver? Se não bate, o que foi perdido ou duplicado na ligação com as dimensões?

P3. Por que a dimensão usa uma chave substituta em vez da chave original da fonte?

P4. O JOIN reproduz o resultado da Gold agregada? Em que situação o modelo dimensional responde algo que a Gold agregada não consegue responder?

P5. O que acontece com a fato se uma dimensão receber um valor novo ou tiver um valor antigo alterado? Como o grupo trataria cada caso?

BLOCO 7 | COSTURAR O PIPELINE

Passos

[ ] Escrevam a função pipeline(), que chama extract, transform e load nessa ordem.

[ ] Antes de cada etapa, verifiquem se a entrada existe. Se não existir, parem com uma mensagem que diga qual etapa anterior está faltando.

[ ] Façam cada etapa gravar uma linha em um CSV de histórico, com etapa, horário, duração em segundos, linhas lidas e linhas escritas. Meçam o tempo com time.perf_counter().

[ ] Testem a dependência: apaguem a Silver e chamem apenas load().

[ ] Prova final: apaguem toda a pasta data, executem pipeline() uma única vez e comparem a Gold nova com a anterior.

[ ] Executem pipeline() duas vezes seguidas e confirmem que o resultado não muda.

Perguntas de verificação (respondam no notebook)

P1. O que aconteceu ao chamar load() sem a Silver? A mensagem de erro ajudou a encontrar o problema?

P2. A Gold reconstruída do zero é idêntica à anterior? Como o grupo verificou isso?

P3. Qual etapa consumiu mais tempo? Quantas linhas por segundo ela processou?

P4. Se o Extract falhar e o Transform rodar mesmo assim sobre a Bronze antiga, o que a Gold vai mostrar e como alguém perceberia o problema?

P5. Desenhem as dependências entre extract, transform e load e escrevam o que deve acontecer com as etapas seguintes quando uma delas falha.

BLOCO 8 | CARGA INCREMENTAL

Passos

[ ] Dividam a Bronze em duas partes usando uma coluna de data (ou faixas de linhas, se não houver data): uma carga inicial com cerca de 80% dos registros e uma carga nova com o restante.

[ ] Reconstruam o pipeline apenas com a carga inicial e anotem o número de linhas de cada camada e o resultado da Gold.

[ ] Acrescentem a carga nova à Bronze sem apagar a anterior e executem o pipeline de novo.

[ ] Garantam que registros que já estavam na Silver não sejam duplicados: definam uma chave única e removam a duplicidade ao combinar o dado novo com o antigo.

[ ] Executem o pipeline mais uma vez, sem nenhuma carga nova, e confirmem que nada muda.

[ ] Comparem a Gold final com a Gold gerada pelo pipeline sobre a base completa, de uma só vez.

Perguntas de verificação (respondam no notebook)

P1. Quantas linhas cada camada tinha após a carga inicial e após a carga nova?

P2. A Gold gerada em duas cargas é igual à gerada de uma só vez? Se houver diferença, de onde ela vem?

P3. Qual chave o grupo usou para impedir duplicidade? O que aconteceria se ela não fosse única de verdade?

P4. O que deveria acontecer na Gold se o dado novo corrigisse um registro que já existia? O pipeline do grupo trata esse caso?

BLOCO 9 | SEGUNDA GOLD E VISUALIZAÇÃO

Passos

[ ] Escrevam uma segunda pergunta de negócio, diferente da primeira, que possa ser respondida pela mesma Silver.

[ ] Criem a segunda tabela Gold, gravem em Parquet e em SQLite e acrescentem-na à função load().

[ ] Construam um gráfico para cada Gold com matplotlib, usando o tipo adequado, com título, rótulos de eixo e unidades.

[ ] Salvem os gráficos como PNG na pasta data/gold/graficos.

[ ] Executem pipeline() do zero e confirmem que as duas Golds e os dois gráficos são gerados novamente.

Perguntas de verificação (respondam no notebook)

P1. A segunda pergunta exigiu algo que a Silver não tinha? Se sim, em qual camada a correção deve ser feita e por quê?

P2. Por que manter duas tabelas Gold em vez de uma única tabela com tudo?

P3. O gráfico escolhido permite que alguém sem conhecimento técnico responda à pergunta em poucos segundos? O que mudaria para melhorá-lo?

P4. Qual das duas Golds é mais cara de recalcular e por quê?

BLOCO 10 | PARAMETRIZAR O PIPELINE

Passos

[ ] Reúnam todos os caminhos, nomes de arquivo e parâmetros (período, filtro de escopo, tamanho de lote, se houver) em um único dicionário de configuração, no topo do notebook.

[ ] Troquem os valores fixos dentro das funções por valores desse dicionário.

[ ] Troquem os print por logging, com nível INFO para o andamento e ERROR para falhas, gravando em um arquivo pipeline.log.

[ ] Executem pipeline() com outro valor de parâmetro (outro período, outro filtro ou outra categoria) sem alterar nenhuma linha das funções.

[ ] Confirmem no log e no histórico de execuções quais parâmetros foram usados em cada execução.

Perguntas de verificação (respondam no notebook)

P1. Quantos pontos do código precisaram ser alterados para trocar o parâmetro? Se foi mais de um, o que ficou fixo dentro das funções?

P2. O que o grupo consegue descobrir lendo apenas o pipeline.log de uma execução que falhou?

P3. Que parâmetro, se alterado por engano, produziria uma Gold errada sem gerar nenhum erro?

P4. Por que registrar os parâmetros junto do histórico de execuções?

BLOCO 11 | DOCUMENTAÇÃO E REPRODUTIBILIDADE

Passos

[ ] Escrevam um README.md na raiz do projeto com: pergunta de negócio, fontes usadas, descrição de cada camada, decisões de transformação, ordem de execução e como rodar o pipeline.
[ ] Criem o requirements.txt com as bibliotecas e as versões usadas.
[ ] Troquem a pasta do projeto, sem a pasta data, com outro grupo.
[ ] Executem o pipeline do outro grupo seguindo apenas o README e anotem cada obstáculo encontrado.
[ ] Devolvam ao grupo dono uma lista de tudo o que impediu ou atrapalhou a execução.

Perguntas de verificação (respondam no notebook)

P1. O pipeline do outro grupo rodou seguindo apenas o README? Que informação faltou?
P2. A Gold gerada no computador de vocês é igual à do grupo dono? Se não for, o que explica a diferença?
P3. Que parte do pipeline do seu grupo só funciona na sua máquina?
P4. O que havia no README do outro grupo que vocês incluiriam no de vocês?

CHECKLIST DE ENTREGA

[ ] Pergunta de negócio escrita no topo do notebook.
[ ] Bronze com todas as fontes, sem alteração, com metadados de extração.
[ ] Silver com as decisões documentadas no formato: decidimos X porque Y.
[ ] Duas tabelas Gold, em Parquet e em SQLite, e um gráfico para cada uma.
[ ] Modelo dimensional (fato e dimensões) gravado, com a soma da fato batendo com a da Silver.
[ ] pipeline() reexecutável: apaga a pasta data, executa e a Gold volta idêntica.
[ ] Carga incremental sem duplicidade de registros.
[ ] Configuração em um único dicionário, pipeline.log e histórico de execuções em CSV.
[ ] README.md e requirements.txt na raiz do projeto.
[ ] Respostas das perguntas de verificação em células de texto do notebook.

Para a próxima aula: tragam a camada Gold pronta e pensem: como o grupo provaria que esse dado é confiável?

DATASETS SUGERIDOS

Critérios para escolher (o dataset precisa atender aos três)

[ ] Possui pelo menos 2 fontes que precisam ser combinadas, OU 1 fonte com mais de 10 mil linhas e problemas de qualidade visíveis.

[ ] Possui uma coluna de data ou de categoria que permita agregar e responder a uma pergunta.

[ ] Pode ser acessado e baixado sem necessidade de cadastro.

Dataset	Formato e tamanho	O que o grupo terá de tratar	Pergunta de negócio possível
D1 Olist, e-commerce brasileiro	Vários CSVs relacionados, cerca de 100 mil pedidos entre 2016 e 2018.	Junção de várias tabelas, nulos com significados diferentes, status de pedido que exige decisão de escopo.	Quais categorias mais faturam por mês? Qual estado tem mais atraso na entrega?
D2 NYC Táxi amarelo, janeiro de 2024, mais zonas	Parquet com quase 3 milhões de viagens (cerca de 50 MB) e um CSV de zonas.	Volume grande (leiam só as colunas necessárias), valores inválidos, nulos, datas fora do período, enriquecimento pela tabela de zonas.	Quais zonas de embarque geram mais receita por hora do dia?
D3 Chinook, loja de músicas	Banco SQLite com 11 tabelas. Cerca de 1 MB.	Fonte é um banco (extração via SQL), várias junções em cadeia, nomes de colunas fora do padrão.	Qual a receita por gênero musical e por país?
D4 Banco Central: Selic, IPCA e dólar	Três séries em JSON, via API, sem chave.	Valores numéricos que chegam como texto, datas em dd/mm/aaaa, séries com frequências diferentes (diária e mensal) que precisam ser alinhadas.	Em quais meses o juro real foi negativo entre 2020 e 2023?
D5 Câmara dos Deputados, API de dados abertos	JSON paginado, até 100 itens por página, sem chave.	Paginação, JSON com campos aninhados, combinação de endpoints diferentes (deputados e partidos).	Quantos deputados cada partido tem em cada estado?
D6 USGS, terremotos do último mês	CSV com cerca de 10 mil linhas e 22 colunas, atualizado continuamente.	Eventos que não são terremotos misturados na base, região escondida em uma coluna de texto, dado que muda a cada dia (o resultado de hoje não será o de amanhã).	Quais regiões tiveram mais eventos de magnitude 4 ou maior?

Fontes para enriquecer qualquer um dos datasets: IBGE municípios (cruzar por cidade ou estado) e Open-Meteo (cruzar por data). O grupo também pode propor outro dataset, desde que o professor valide os três critérios antes de o bloco 3 começar.

Links de acesso

D1 Olist, página com todos os arquivos: https://huggingface.co/datasets/bulutttt/olist-raw-data/tree/main

D1 Olist, modelo de endereço de download (troquem o nome do arquivo): https://huggingface.co/datasets/bulutttt/olist-raw-data/resolve/main/olist_orders_dataset.csv

D1 Olist, espelho alternativo: https://huggingface.co/datasets/MafiaAzulBr/olistcsv/tree/main/archive

D2 NYC Táxi, viagens de janeiro de 2024: https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2024-01.parquet

D2 NYC Táxi, tabela de zonas: https://d37ci6vzurychx.cloudfront.net/misc/taxi_zone_lookup.csv

D3 Chinook, banco SQLite: https://raw.githubusercontent.com/lerocha/chinook-database/master/ChinookDatabase/DataSources/Chinook_Sqlite.sqlite

D4 Banco Central, Selic (série 432): https://api.bcb.gov.br/dados/serie/bcdata.sgs.432/dados?formato=json&dataInicial=01/01/2020&dataFinal=31/12/2023

D4 Banco Central, IPCA (série 433): https://api.bcb.gov.br/dados/serie/bcdata.sgs.433/dados?formato=json&dataInicial=01/01/2020&dataFinal=31/12/2023

D4 Banco Central, dólar (série 1): https://api.bcb.gov.br/dados/serie/bcdata.sgs.1/dados?formato=json&dataInicial=01/01/2020&dataFinal=31/12/2023

D5 Câmara, deputados: https://dadosabertos.camara.leg.br/api/v2/deputados?itens=100&formato=json

D5 Câmara, partidos: https://dadosabertos.camara.leg.br/api/v2/partidos?itens=100&formato=json

D6 USGS, terremotos do último mês: https://earthquake.usgs.gov/earthquakes/feed/v1.0/summary/all_month.csv

+ IBGE, municípios brasileiros (para enriquecer por cidade ou estado): https://servicodados.ibge.gov.br/api/v1/localidades/municipios

+ Open-Meteo, clima histórico diário (para enriquecer por data): https://archive-api.open-meteo.com/v1/archive?latitude=-24.72&longitude=-53.74&start_date=2023-01-01&end_date=2023-12-31&daily=temperature_2m_max,precipitation_sum&timezone=America/Sao_Paulo